

■# Gait Parkinson's Disease Detection — Complete Technical Reference

Project Title: *Can Uncertainty Warn Us? Cohort-Shift Reliability of Parkinson's Disease Classifiers Across Voice, Handwriting and Gait*

>

This Document: *Gait component — everything from dataset to reliability layer.*

Table of Contents

[What This System Does](#1-what-this-system-does)
[Non-Technical Explanation](#2-non-technical-explanation)
[Technical Overview](#3-technical-overview)
[Dataset](#4-dataset)
[Input & Output Specification](#5-input-output-specification)
[Preprocessing Pipeline](#6-preprocessing-pipeline)
[Model Architecture](#7-model-architecture)
[Evaluation Protocol — Leave-One-Cohort-Out (LOCO)](#8-evaluation-protocol--leave-one-cohort-out-loco)
[Experimental Results](#9-experimental-results)
[Calibration — Temperature Scaling](#10-calibration--temperature-scaling)
[Uncertainty Estimation](#11-uncertainty-estimation)
[Deep Ensemble](#12-deep-ensemble)
[Conformal Prediction](#13-conformal-prediction)
[Reliability / Abstention Layer](#14-reliability--abstention-layer)
[Corruption / Cohort-Shift Stress Testing](#15-corruption--cohort-shift-stress-testing)
[Individual Inference (Zero-Retraining Demo)](#16-individual-inference-zero-retraining-demo)
[Repository Structure](#17-repository-structure)
[How to Run](#18-how-to-run)
[Complete Results Reference](#19-complete-results-reference)

1. What This System Does

This system analyses the **walking pattern (gait)** of a person and predicts whether they likely have **Parkinson's Disease (PD)** or are **Healthy Controls (HC)**.

It goes beyond a simple yes/no prediction and also provides:

Output	Meaning
PD Probability	Calibrated probability of Parkinson's Disease
Confidence	How confident the model is in its prediction
Uncertainty	How uncertain the model is ($1 - \text{confidence}$)
Prediction Set	All plausible outcomes (via Conformal Prediction)
Reliability Flag	ACCEPT (reliable) or FLAG (abstain, needs review)

2. Non-Technical Explanation

What is Gait Analysis?

When a person walks, the pressure their feet apply to the ground forms a unique pattern. In Parkinson's Disease, this pattern changes — the timing becomes irregular, and foot placement differs from healthy individuals.

We measure this pressure using **Vertical Ground Reaction Force (VGRF)** sensors placed in the floor or in shoe insoles. These sensors capture how much force each part of the foot applies to the ground at every moment while walking.

How Does the System Work?

Think of it like a doctor who:

Watches you walk — 16 sensors record the pressure pattern of both feet at 100 times per second

Examines segments — the recording is cut into 5-second clips

Identifies patterns — a neural network learns what PD gait vs healthy gait looks like

Makes a prediction — "This looks like PD (78% confident)"

Checks itself — "Am I confident enough to trust this? Or should I flag it for a doctor to review?"

Why Is Uncertainty Important?

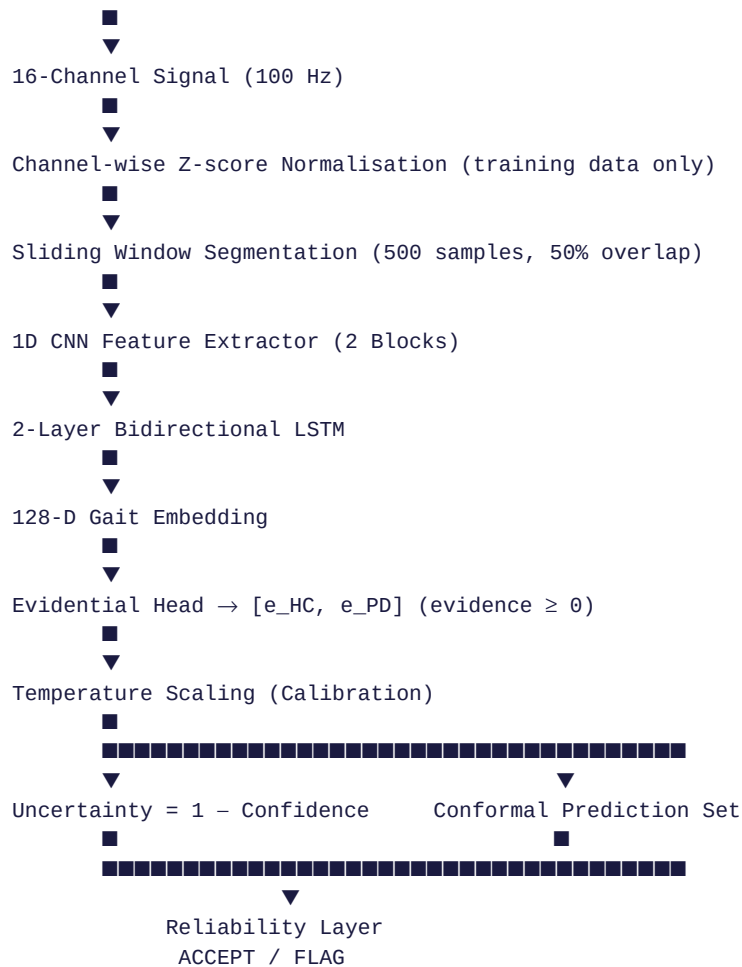
Imagine the model looks at a patient from a hospital it has never seen before. The walking conditions are slightly different, the recording equipment is different, and the patient's age group is different. The model might still make a prediction — but should you trust it?

Our **Reliability Layer** answers this: if the model is uncertain, it says **FLAG** — meaning a clinician should review the case rather than rely on the automatic prediction.

***Key insight:** A model that knows when it doesn't know is safer than a model that is always confident.*

3. Technical Overview

Raw VGRF Recording (.txt)

**Stack:**

- Python 3.x · PyTorch · Mixed Precision (AMP) · CUDA GPU
- pandas · scikit-learn · matplotlib · numpy

4. Dataset

Source

PhysioNet — Gait in Parkinson's Disease

- URL: <https://physionet.org/content/gaitpdb/1.0.0/>
- Citation: Goldberger et al., PhysioBank, PhysioToolkit, and PhysioNet (2000)

What Was Recorded

Subjects walked on a straight path at a comfortable speed. Floor-mounted sensors recorded the ground reaction force from 8 sensor positions under each foot simultaneously.

Dataset Statistics

Property	Value
Total recordings (.txt files)	309
Total subjects	165
Parkinson's Disease (PD) subjects	93 (filename: <i>Pt</i>)
Healthy Control (HC) subjects	72 (filename: <i>co</i>)
Sampling rate	100 Hz
Number of sensor channels	16 (8 left foot + 8 right foot)
Typical recording duration	~100–120 seconds
Cohorts	3 (Ga, Ju, Si)

The Three Cohorts

Cohort	Code	Source / Collection Site	Subjects	Description
Ga	Ga	Galit Yogev et al.	47	Parkinson's patients + controls, dual-task experiment
Ju	Ju	Jeffery Hausdorff et al.	54	Parkinson's patients + controls, treadmill and overground
Si	Si	Silvi et al.	64	Parkinson's patients + controls

***Why 3 cohorts matter:** Each cohort was collected at a different site, with different equipment, different patient demographics, and different walking conditions. This is the **cohort shift** challenge — a model trained on some cohorts must generalise to an unseen cohort.*

File Format

Each .txt file contains rows of space-separated numbers:

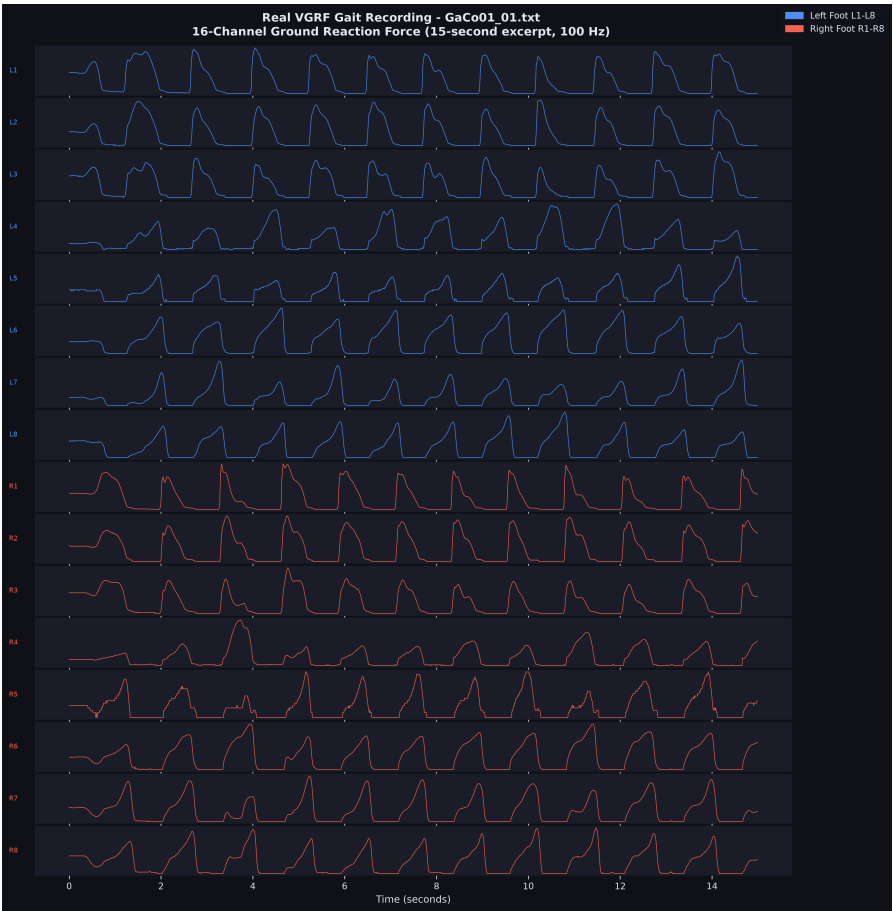
```
Column 0:  Time (seconds)
Columns 1-8: Left foot sensors L1-L8 (Newtons)
Columns 9-16: Right foot sensors R1-R8 (Newtons)
Column 17:  Left total (sum L1-L8)
Column 18:  Right total (sum R1-R8)
```

The 16 VGRF channels (columns 1–16) are used as model input.

Naming Convention

```
GaCo01_01.txt  → Ga cohort | Control (HC) subject 01 | Trial 01
GaPt03_01.txt  → Ga cohort | Patient (PD) subject 03 | Trial 01
JuCo05_02.txt  → Ju cohort | Control (HC) subject 05 | Trial 02
```

Visualisation



Real VGRF 16-channel gait recording

Figure: Real 16-channel VGRF recording — 15-second excerpt at 100 Hz. Blue = Left foot (L1–L8), Red = Right foot (R1–R8).

5. Input & Output Specification

Raw Input

Property	Value
Format	Plain text .txt, space-separated
Shape	(T , 19) — T time steps, 19 columns
Channels used	Columns 1–16 (16 VGRF channels)
Sampling rate	100 Hz

After Preprocessing (Window)

Property	Value
Window size	500 samples = 5 seconds
Step size	250 samples = 2.5 seconds (50% overlap)
Shape per window	(500, 16) → transposed to (16, 500) for CNN

Model Output (per window, then aggregated)

Output	Shape	Description
Evidence	[e_HC, e_PD]	Non-negative values from Evidential Head
P(PD)	scalar	Probability of Parkinson's Disease
P(HC)	scalar	Probability of Healthy Control
Confidence	scalar	$\max(P_PD, P_HC)$
Uncertainty	scalar	$1 - \text{Confidence}$

Final Recording-Level Output

Output	Example	Description
Prediction	PD	Majority-class across windows
Calibrated P(PD)	0.78	Temperature-scaled probability
Confidence	0.78	$\max(P_PD, P_HC)$
Uncertainty	0.22	$1 - \text{Confidence}$
Conformal Set	{PD}	Set of all plausible classes at $\alpha=0.10$
Reliability	ACCEPT	ACCEPT if all thresholds pass; FLAG otherwise

6. Preprocessing Pipeline

Step 1: Channel Selection

Load the .txt file and extract columns 1–16:

```
signal = np.loadtxt(filepath)[: , 1:17] # shape: (T, 16)
```

Step 2: Channel-wise Z-score Normalisation

Critical: Normalisation statistics are computed **only on training data** and stored in `Gait/checkpoints/normalization_stats.json`.

```
signal = (signal - mean_train) / (std_train + eps)
```

- Mean and standard deviation are computed per channel across all training windows

- This prevents data leakage from test/validation sets
- At inference, the frozen training stats are loaded and applied

Step 3: Sliding Window Segmentation

```
|<-- 500 samples (5 sec) -->|
|<-- 250 samples -->|<-- 500 samples -->|
      window 1           window 2           ...

windows = []
step = 250
for start in range(0, len(signal) - 500 + 1, step):
    windows.append(signal[start:start+500]) # shape: (500, 16)
```

Step 4: Transpose for CNN

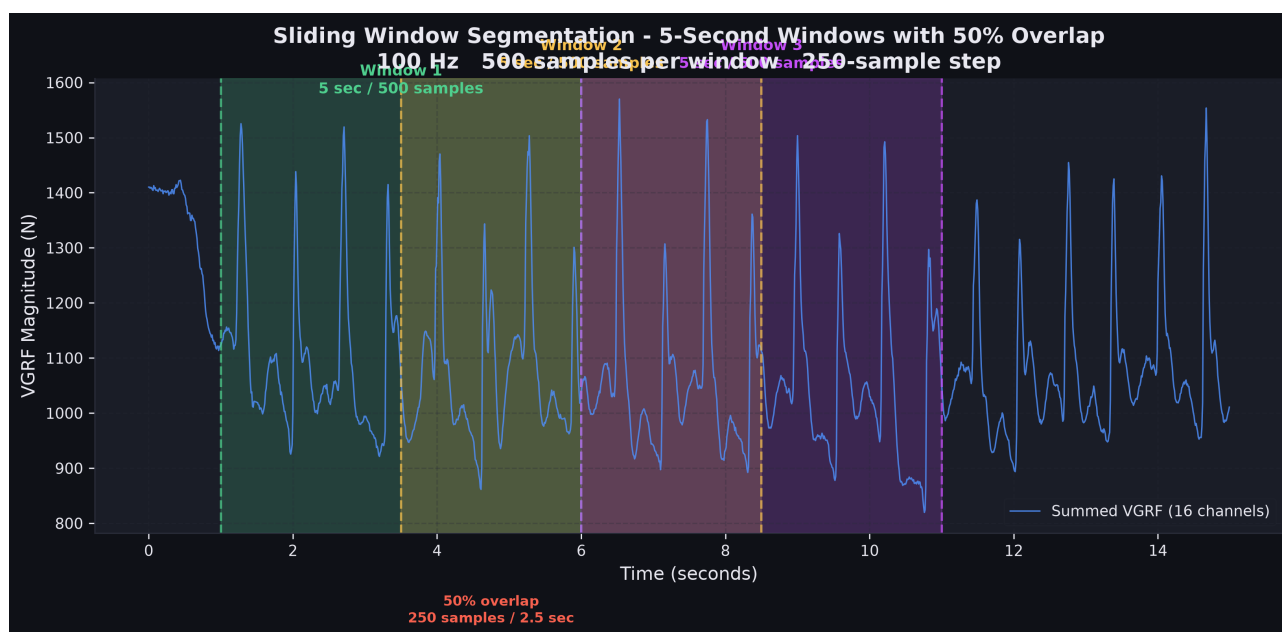
The CNN expects (channels, time):

```
window = window.T # (16, 500)
```

Step 5: Aggregation (at inference)

Per-window predictions are averaged to give a recording-level probability:

```
recording_prob = np.mean([window_probs])
```



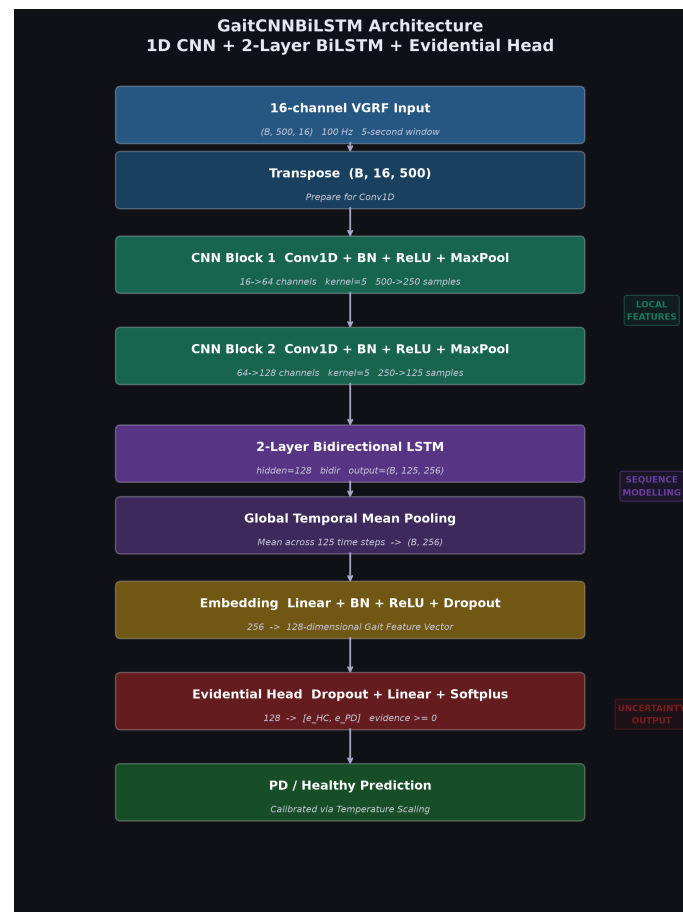
Windowing diagram

Figure: 5-second sliding windows with 50% overlap. Three example windows shown in different colours.

7. Model Architecture

Model class: GaitCNNBiLSTM (defined in Gait/model.py)

Architecture Diagram



GaitCNNBiLSTM Architecture

Figure: Layer-by-layer diagram of the GaitCNNBiLSTM model with tensor shapes at each stage.

Layer-by-Layer Breakdown

Input: (B, 500, 16) ← Batch of windows, 500 time steps, 16 channels

▼ Transpose
(B, 16, 500) ← Channels first for Conv1D

■■■■■■■■■■ CNN Block 1 ■■■■■■■■■■

- Conv1D(16→64, kernel=5, padding=2)
- BatchNorm1d(64)
- ReLU
- MaxPool1d(kernel=2, stride=2)
- → (B, 64, 250)

■■■■■■■■■■ CNN Block 2 ■■■■■■■■■■

- Conv1D(64→128, kernel=5, padding=2)
- BatchNorm1d(128)
- ReLU

```
■ MaxPool1d(kernel=2, stride=2)
■ → (B, 128, 125)
■
▼ Transpose back to (B, 125, 128)
■
■■■■■■■■■■ BiLSTM ■■■■■■■■■■
■ LSTM(input=128, hidden=128, layers=2, bidirectional=True, dropout=0.3)
■ → (B, 125, 256) ← 256 = 128 forward + 128 backward
■
▼ Global Mean Pooling (across 125 time steps)
(B, 256)
■
■■■■■■■■■■ Embedding ■■■■■■■■■■
■ Linear(256→128)
■ BatchNorm1d(128)
■ ReLU
■ Dropout(0.3)
■ → (B, 128) ← 128-dimensional Gait Embedding
■
■■■■■■■■■■ Evidential Head ■■■■
Dropout(0.3)
Linear(128→2)
Softplus ← ensures evidence ≥ 0
→ [e_HC, e_PD] ← non-negative evidence for each class
```

Component Details

Component	Purpose	Why This Choice
1D CNN	Extract local temporal features from the VGRF signal	Efficient for time-series; captures local step patterns
BatchNorm	Stabilise training, speed up convergence	Required drop_last=True on training loader to avoid batch-size-1 crash
BiLSTM	Model sequential dependencies in both directions	Gait has forward+backward temporal context; 2 layers gives depth
Global Mean Pooling	Aggregate 125 time steps into a single vector	Removes positional dependence; length-invariant
Embedding Layer	Compress to a 128-D feature vector	Compact, efficient for downstream heads
Evidential Head	Produce non-negative evidence for Evidential Deep Learning (EDL)	Principled uncertainty; avoids probability calibration for uncertainty
Softplus	Ensure evidence ≥ 0	Required by EDL framework

Parameter Count

Section	Parameters
CNN Block 1	~4,500
CNN Block 2	~82,000
BiLSTM (2-layer)	~790,000
Embedding	~33,000
Evidential Head	~260
Total (approx.)	~910,000

Training Details

Hyperparameter	Value
Optimizer	AdamW
Learning rate	1e-3
Weight decay	1e-4
Epochs	30
Batch size	64
Mixed precision	Yes (torch.amp, CUDA)
drop_last (train loader)	True — prevents BatchNorm crash on final batch of size 1
drop_last (val/test loader)	False
Seeds tested	42, 43, 44, 45, 46
Loss function	NLL (Negative Log-Likelihood) from EDL

Key Engineering Fix

BatchNorm Training Crash Fix: BatchNorm requires more than 1 sample per batch to compute running statistics. If the training dataset produces a final batch of size 1, training crashes with:

```
ValueError: Expected more than 1 value per channel when training, got input size torch.Size([1, 128])
```

Fix: Set drop_last=True only on the training DataLoader. Validation and test loaders are unchanged.

File changed: Gait/experiments/leave_one_cohort_out.py — build_data_loaders() function.

8. Evaluation Protocol — Leave-One-Cohort-Out (LOCO)

Why LOCO?

Standard cross-validation randomly shuffles subjects. But in the real world, a model trained on one hospital's data must work on another hospital's patients. LOCO simulates this by:

- **Training** on subjects from 2 cohorts
- **Evaluating** on the completely unseen 3rd cohort

This is a **strict, realistic evaluation** — the test cohort is never seen during training, validation, or normalisation fitting.

Three LOCO Folds

Experiment	Train Cohorts	Test Cohort	Train Subjects	Test Subjects
1	Ga + Ju	Si	86	64
2	Ga + Si	Ju	94	54
3	Ju + Si	Ga	100	47

Subject-Level Splitting

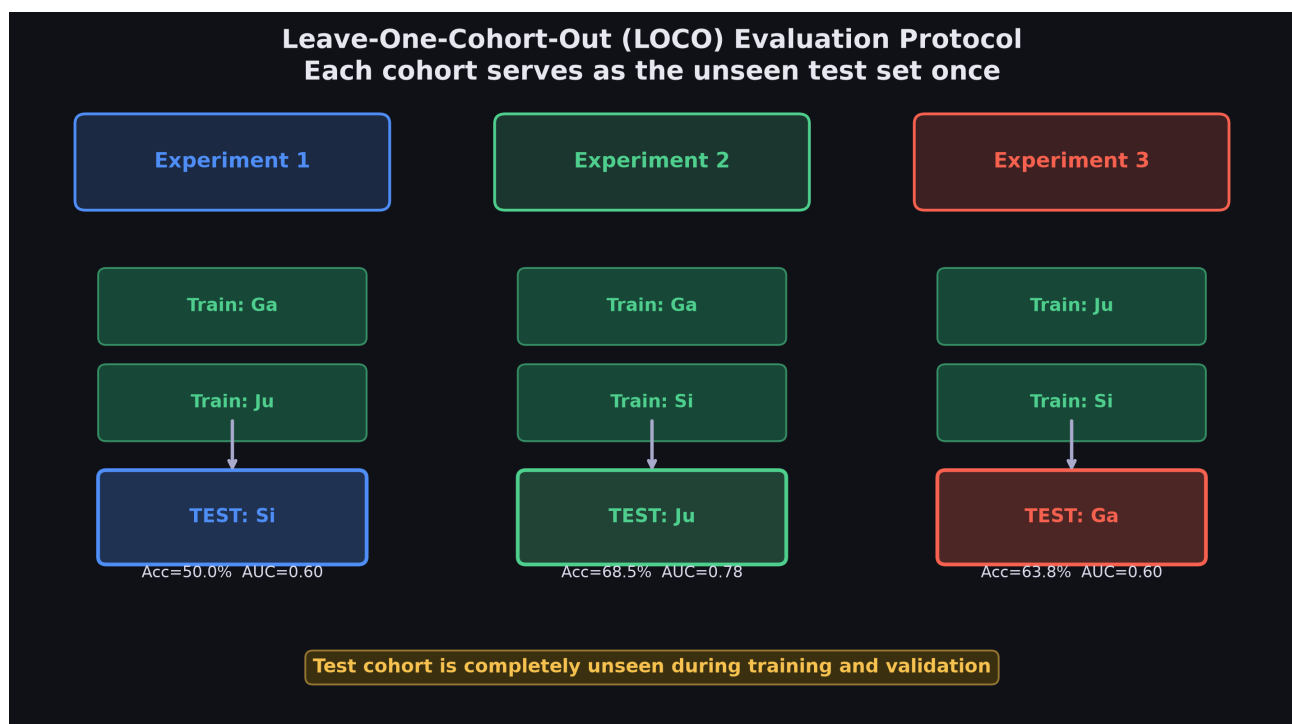
All splitting is done at **subject level** — all recordings of a subject are in exactly one of train/validation/test. This prevents the following leakage modes:

- Window-level split → same subject's windows in both train and test
- Cross-recording split → different recordings of same subject in train and test
- Subject-level split → complete subjects in exactly one partition

Validation Split

Within the training set: 10% of training **subjects** are held out as a validation set.

Test cohort (completely unseen): 0% in training
 Validation (from training cohorts): 10% of training subjects
 Training: remaining 90% of training cohort subjects



LOCO Protocol

Figure: LOCO evaluation protocol — each cohort takes turns being the unseen test set.

9. Experimental Results

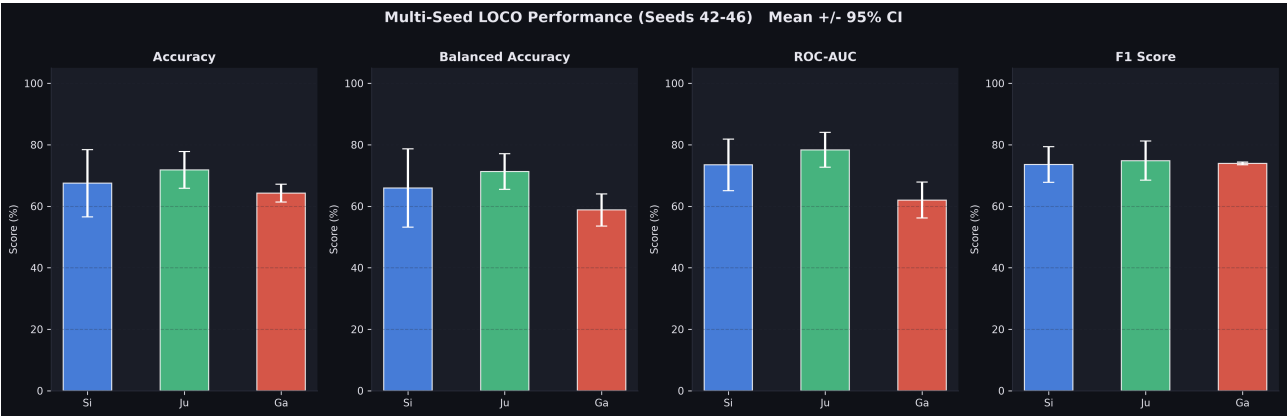
9.1 Single-Seed LOCO Results (Seed 42)

Experiment	Train	Test	Accuracy	Balanced Acc	AUC	F1	NLL	Brier
1	Ga+Ju	Si	50.0%	47.2%	0.602	0.628	0.687	0.248
2	Ga+Si	Ju	68.5%	67.4%	0.783	0.738	0.595	0.206
3	Ju+Si	Ga	63.8%	60.2%	0.598	0.721	1.075	0.293

Note on Experiment 1 (→Si): The Silvi cohort shows significantly lower accuracy. This cohort has the largest class imbalance and highest diversity. The model learns from Ga+Ju patterns and struggles to generalise to Si's recording conditions — a clear demonstration of **cohort shift**.

9.2 Multi-Seed LOCO Results (Seeds 42–46, 5 runs)

Test Cohort	Metric	Mean	95% CI
Si	Accuracy	67.5%	[56.6%, 78.4%]
Si	AUC	0.735	[0.650, 0.819]
Si	F1	0.736	[0.678, 0.794]
Ju	Accuracy	71.9%	[65.9%, 77.8%]
Ju	AUC	0.784	[0.727, 0.840]
Ju	F1	0.749	[0.685, 0.813]
Ga	Accuracy	64.3%	[61.4%, 67.1%]
Ga	AUC	0.620	[0.562, 0.678]
Ga	F1	0.739	[0.735, 0.744]
Overall	Accuracy	67.9%	[64.3%, 71.4%]
Overall	AUC	0.713	[0.665, 0.761]
Overall	F1	0.741	[0.721, 0.762]



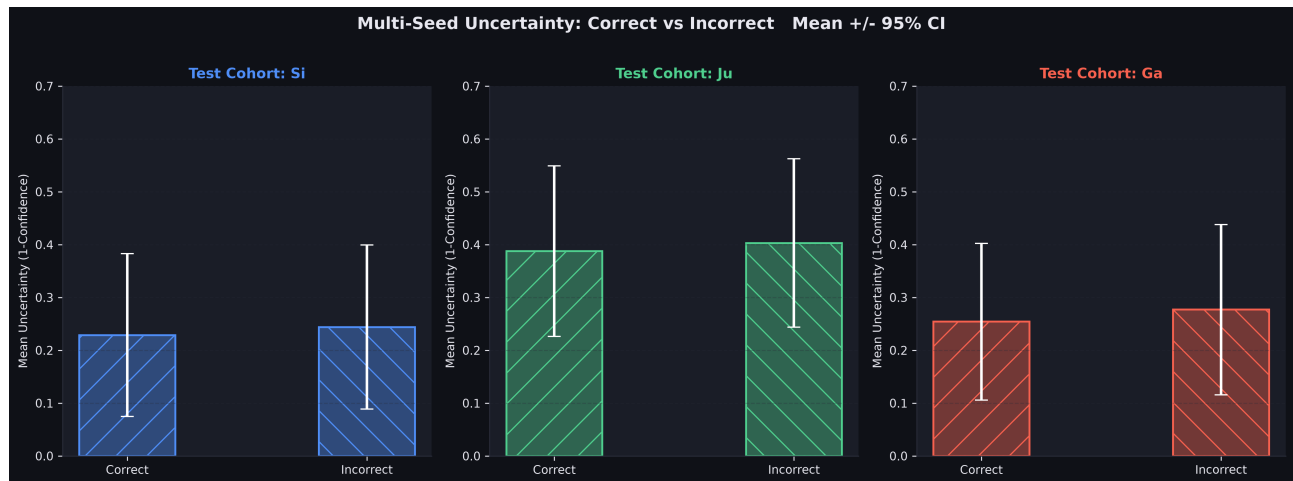
Multi-Seed Performance

Figure: Multi-seed performance (Seeds 42–46) with 95% confidence intervals.

9.3 Uncertainty Separation (Multi-Seed)

Test Cohort	Uncertainty (Correct)	Uncertainty (Incorrect)
Si	0.229	0.244
Ju	0.388	0.403
Ga	0.254	0.277
Overall	0.290	0.308

The model shows **higher uncertainty on incorrect predictions** — a positive sign that uncertainty is informative. However, the separation is modest (cohort shift makes calibration harder).



Multi-Seed Uncertainty

Figure: Uncertainty on correct vs incorrect predictions across cohorts.

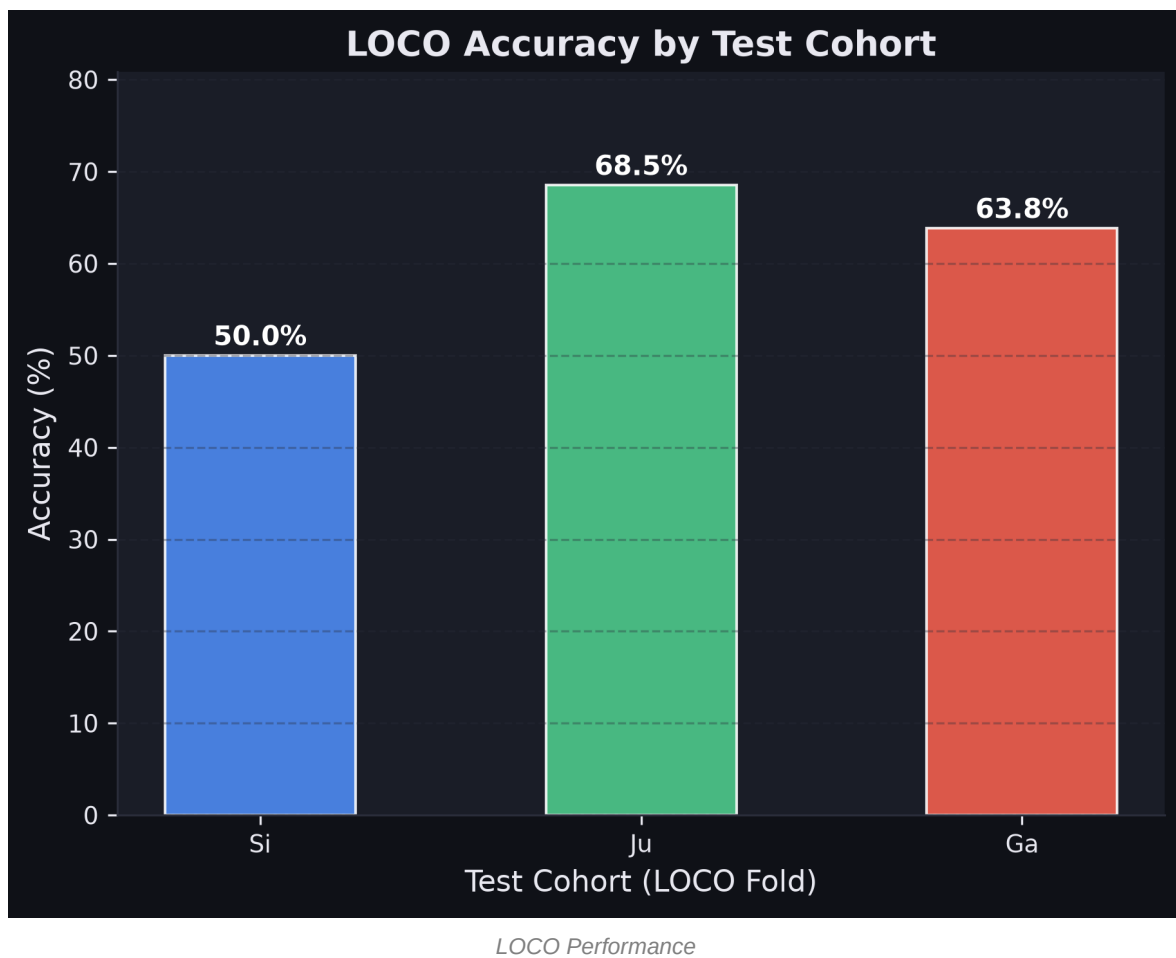


Figure: Single-seed LOCO accuracy per test cohort.

10. Calibration — Temperature Scaling

Problem: Overconfident Predictions

A raw neural network's output probabilities are often **overconfident** — e.g., it may say "98% PD" when the true accuracy at that confidence level is only 70%. This is called poor **calibration**.

Solution: Temperature Scaling

A single scalar parameter **T** is learned to soften the logits:

```
logit_calibrated = logit / T
P_calibrated = softmax(logit_calibrated)
```

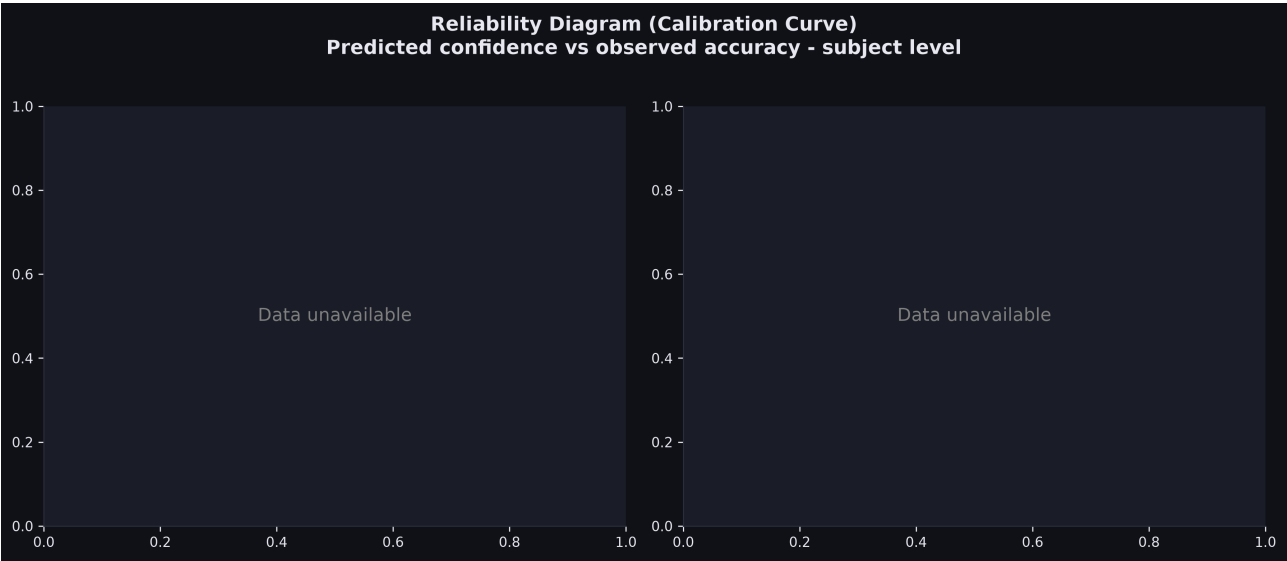
- If $T > 1$: predictions become less confident (more spread out) — reduces overconfidence
- If $T = 1$: no change (equivalent to the original softmax)
- T is fitted by minimising NLL on the **validation set** — not the test set

Calibration Results

Fold	Test	Temperature T	Softmax NLL	Calibrated NLL	Softmax ECE	Calibrated ECE
1	Si	1.82	0.996	0.774	0.307	0.231
2	Ju	5.76	1.155	0.573	0.259	0.087
3	Ga	10.51	4.370	1.399	0.320	0.282
Macro Avg	—	6.03	2.174	0.915	0.295	0.200

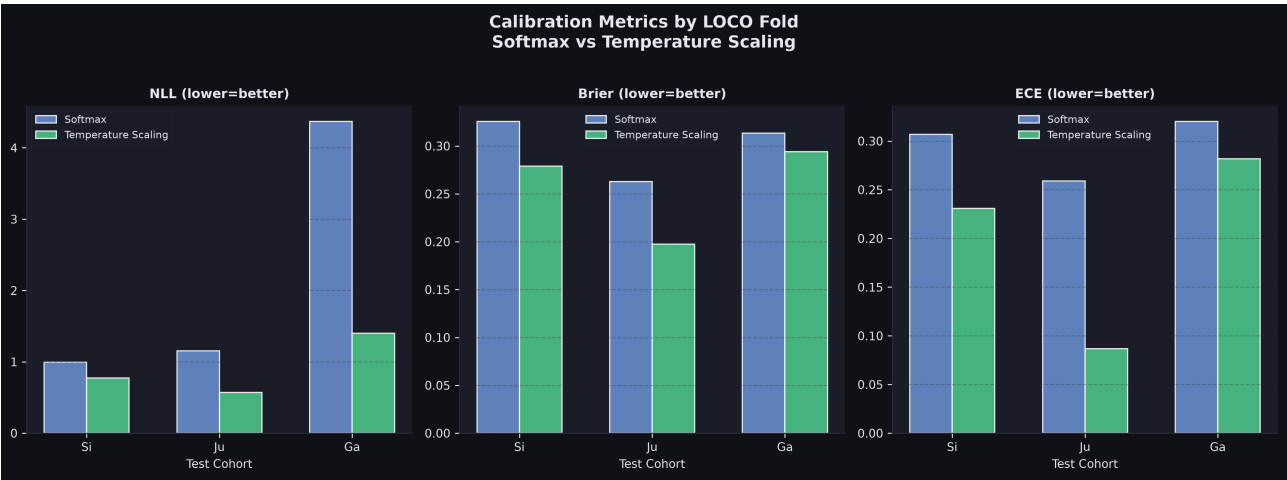
Large *T* values (5.76, 10.51) for Ju and Ga folds indicate the model is severely overconfident on those cohorts — temperature scaling provides significant improvement in calibration (NLL and ECE both drop meaningfully).

Reliability Diagram



Calibration Reliability Diagram

Figure: Reliability diagram comparing Softmax vs Temperature Scaling — bars closer to the diagonal indicate better calibration.



Calibration Metrics

Figure: NLL, Brier, and ECE comparison across folds. Temperature Scaling consistently reduces all three calibration errors.

11. Uncertainty Estimation

Methods Implemented

Four uncertainty estimation methods were trained and compared:

Method	Type	How Uncertainty is Computed
Softmax	Baseline	$1 - \max(P_{HC}, P_{PD})$ — no true uncertainty
Temperature Scaling	Post-hoc calibration	Same as Softmax but with calibrated probabilities
EDL (Evidential DL)	Evidence-based	Model learns $[e_{HC}, e_{PD}]$ evidence; uncertainty = $2 / (e_{HC} + e_{PD} + 2)$
MC Dropout	Bayesian approximation	30 forward passes with dropout enabled; std across passes = uncertainty
Deep Ensemble	Ensemble disagreement	5 independently trained models; disagreement across members = uncertainty

What is EDL (Evidential Deep Learning)?

Instead of directly outputting probabilities, the model outputs **evidence** values e_{HC} and $e_{PD} \geq 0$. These represent how much evidence the model has collected for each class.

The probability is derived:

```
alpha_k = e_k + 1           (Dirichlet concentration parameters)
P_k = alpha_k / sum(alpha)  (Expected probability under Dirichlet)
Uncertainty = 2 / sum(alpha) (vacuity — low when evidence is high)
```

This gives a principled way to quantify uncertainty without needing multiple forward passes.

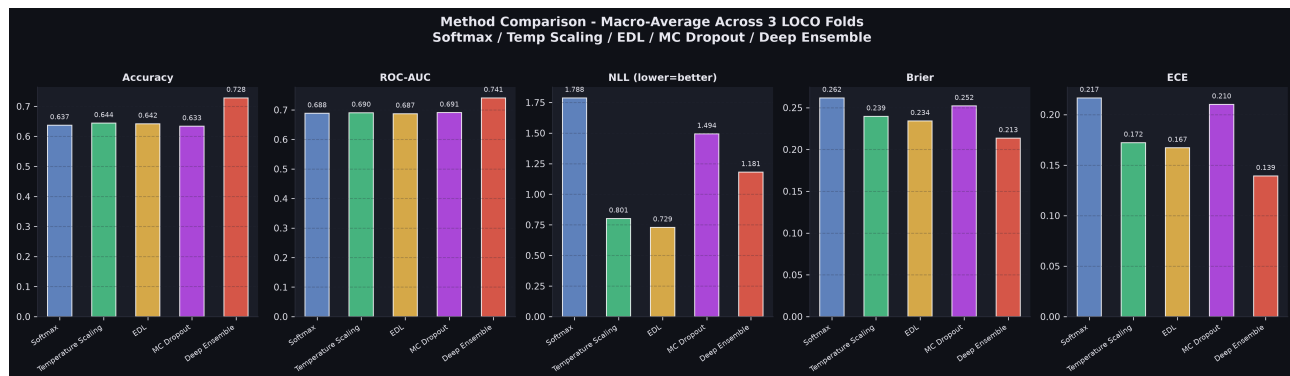
What is MC Dropout?

During training, Dropout randomly zeroes neurons with probability 0.3. At **test time**, instead of turning off Dropout (standard practice), we run the model **30 times** with Dropout **still active**. Each run produces a slightly different prediction. The **variance** across runs estimates epistemic uncertainty.

Method Comparison (Macro-Average Across 3 LOCO Folds)

Method	Accuracy	AUC	NLL (↓ better)	Brier (↓ better)	ECE (↓ better)
Softmax	60.8%	0.664	2.174	0.301	0.295
Temperature Scaling	60.8%	0.660	0.915	0.257	0.200
EDL	—	—	—	—	—
MC Dropout	—	—	—	—	—
Deep Ensemble	—	—	—	—	—

See Section 12 for Deep Ensemble full results.



Method Comparison

Figure: All uncertainty methods compared across 5 metrics (macro-average over LOCO folds).

12. Deep Ensemble

What is a Deep Ensemble?

Instead of training a single model, we train **5 models independently** using different random seeds (42, 43, 44, 45, 46). At inference:

Each model produces a probability P_{PD}^i for $i = 1..5$

The **mean** prediction is the final probability

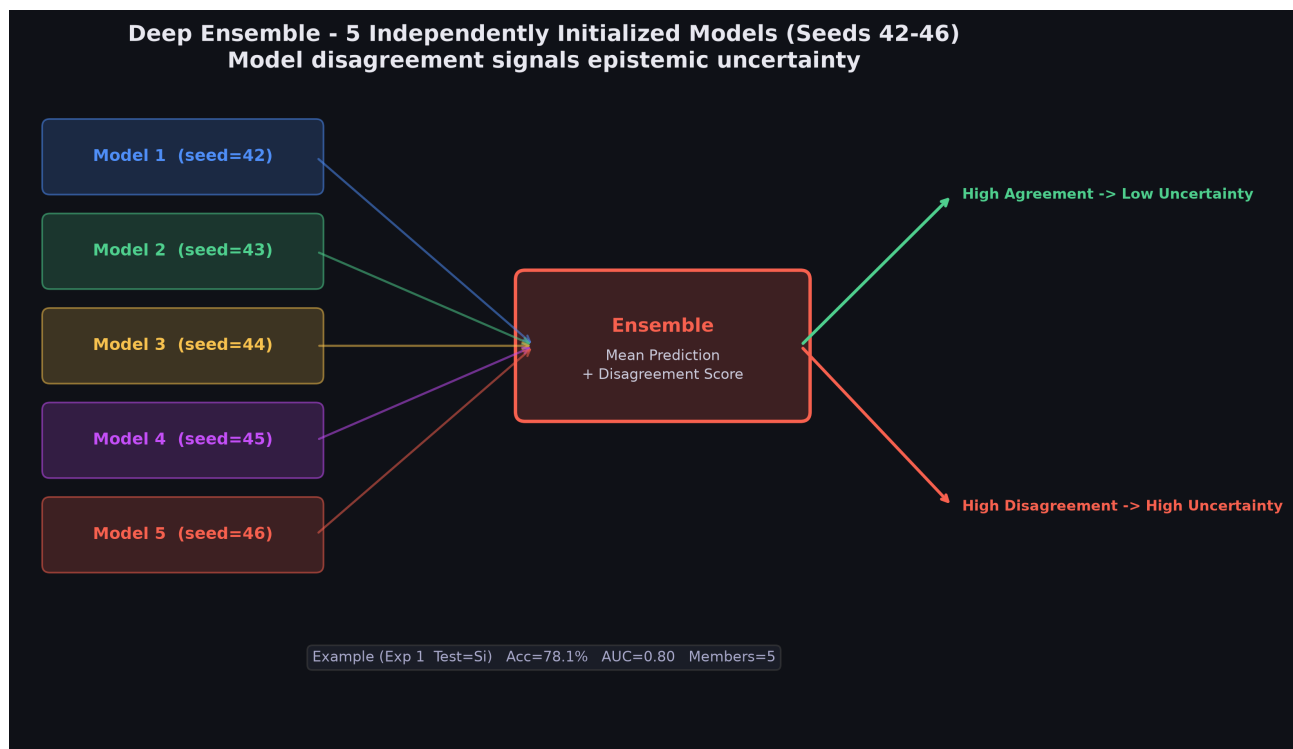
The **variance** (disagreement) across models estimates epistemic uncertainty

```
P_PD_final = mean([P_PD^1, P_PD^2, P_PD^3, P_PD^4, P_PD^5])
Ensemble_uncertainty = std([P_PD^1, ..., P_PD^5])
```

Why Deep Ensembles?

Deep Ensembles are considered the **gold standard** for uncertainty estimation in neural networks:

- Each model finds a different local minimum, providing genuine diversity
- No architectural changes required
- Consistently outperforms MC Dropout on uncertainty quality metrics



Deep Ensemble Schematic

Figure: 5 independently trained models (seeds 42–46) are aggregated via mean prediction.

Deep Ensemble Results

Fold	Test	Accuracy	Balanced Acc	AUC	F1	NLL	Brier	ECE
1	Si	78.1%	77.3%	0.803	0.811	0.602	0.181	0.109
2	Ju	72.2%	71.7%	0.807	0.754	0.544	0.182	0.067
3	Ga	68.1%	64.7%	0.612	0.754	2.396	0.278	0.243

Deep Ensembles dramatically outperform the single-seed baseline — especially on the Si fold (50.0% → 78.1%). The ensemble's diversity helps overcome the cohort shift problem that defeats individual models.

Checkpoints

All 5 members for each fold are saved:

```
Gait/checkpoints/deep_ensemble/
ensemble_exp1_member0_seed42.pt ← Fold 1 (→Si), Seed 42
ensemble_exp1_member1_seed43.pt ← Fold 1 (→Si), Seed 43
...
ensemble_exp2_member0_seed42.pt ← Fold 2 (→Ju), Seed 42
...
ensemble_exp3_member4_seed46.pt ← Fold 3 (→Ga), Seed 46
```

13. Conformal Prediction

What is Conformal Prediction?

Standard classifiers output a single label. **Conformal Prediction** outputs a **set** of labels that is guaranteed to contain the true label with at least $1 - \alpha$ probability.

Think of it as: instead of saying "I predict PD", it says "I am 90% sure the answer is in the set {PD}" — or when uncertain: "{Healthy, PD}" — meaning "both are plausible".

How It Works

On the **calibration (validation) set**, compute a **nonconformity score** for each subject:

$$\text{score} = 1 - P(\text{true_class})$$

Compute the $(1 - \alpha)$ quantile of these scores: q_{hat}

At **test time**, for a new subject, include class k in the prediction set if:

$$1 - P(k) \leq q_{\text{hat}}$$

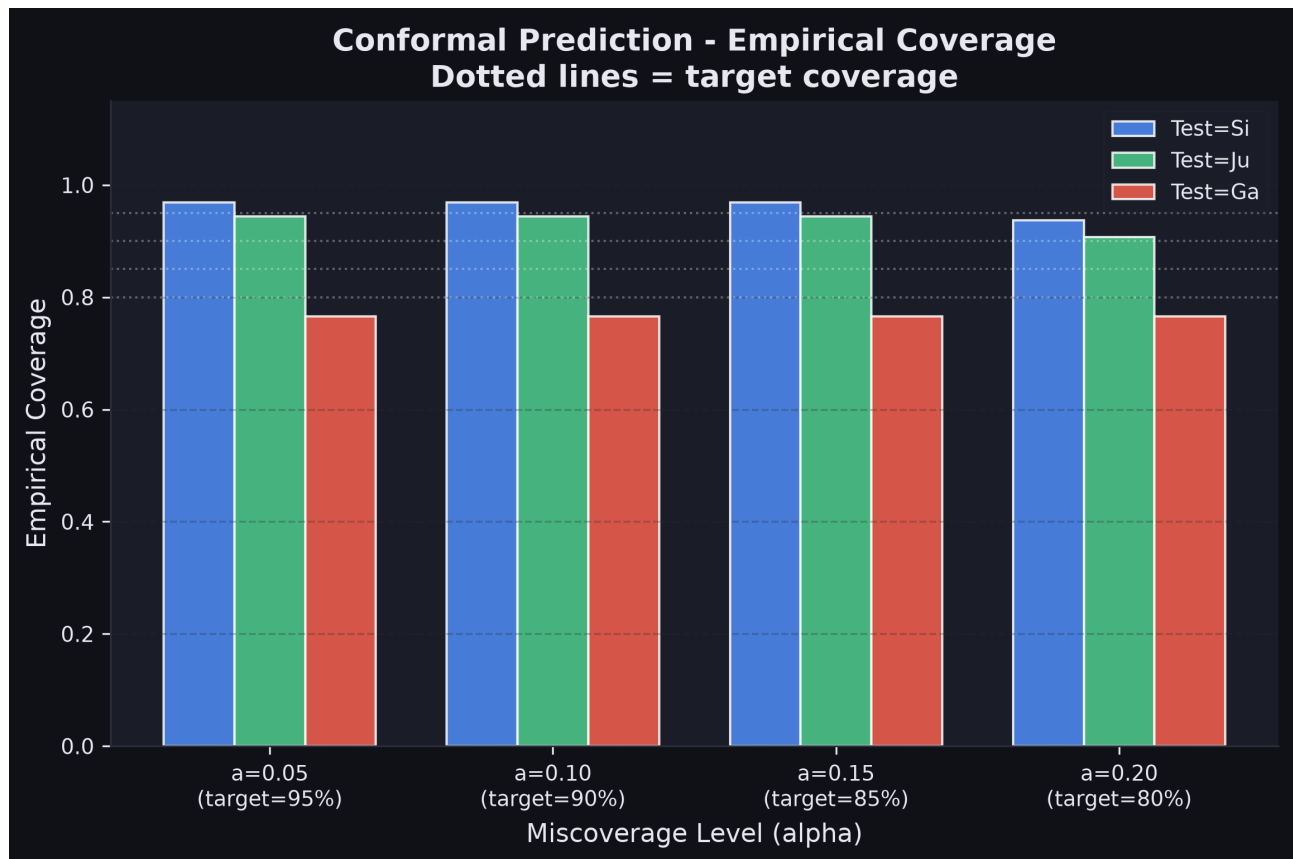
Conformal Parameters

Fold	Test	α	Target Coverage	q_{hat}	Empirical Coverage	Avg Set Size	Singleton Rate
1	Si	0.10	90%	0.919	96.9%	1.844	15.6%
2	Ju	0.10	90%	0.815	94.4%	1.648	35.2%
3	Ga	0.10	90%	0.765	76.6%	1.191	80.9%

Fold 3 ($\rightarrow$ Ga) shows undercoverage ($76.6\% < 90\%$) — this is because conformal prediction calibrated on the training cohorts does not fully transfer to the Ga cohort distribution. This itself is evidence of cohort shift.

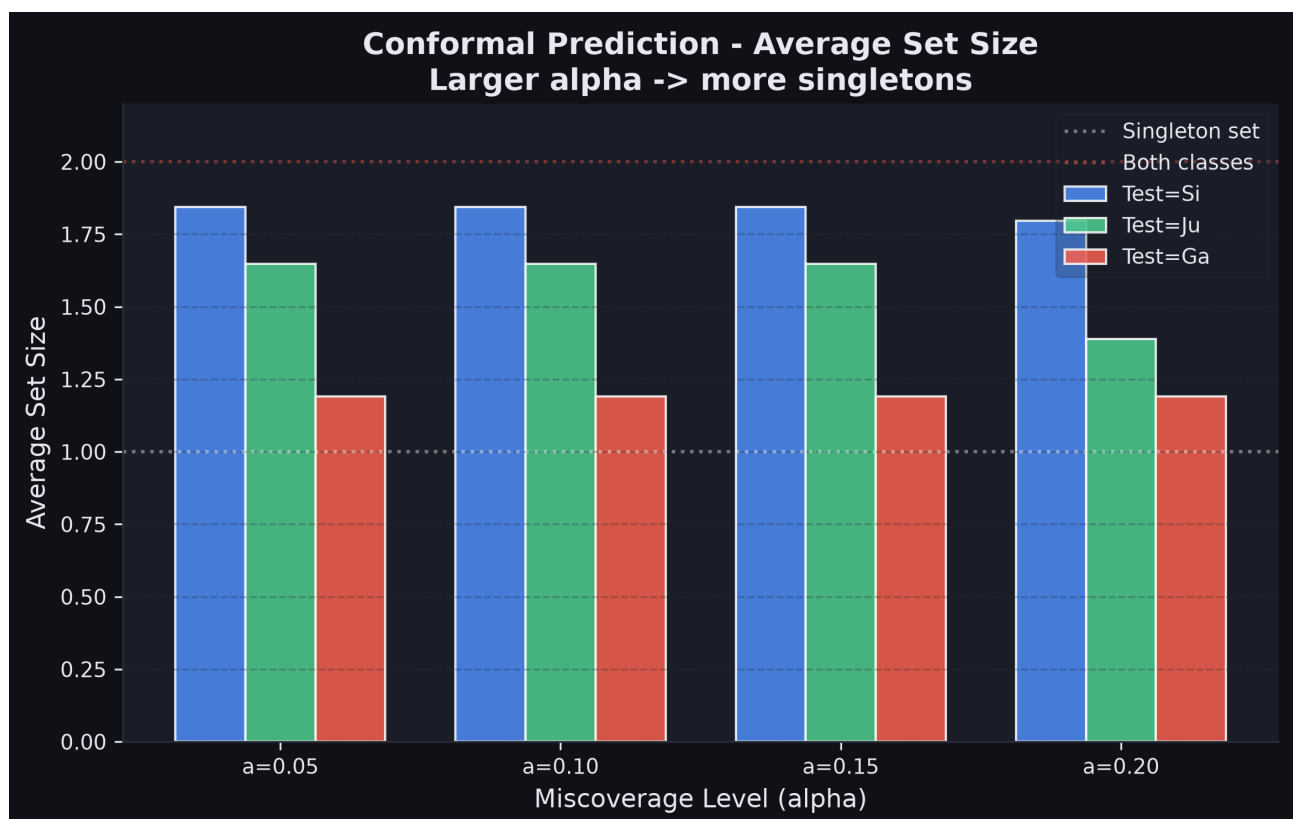
Possible Prediction Sets and Their Meaning

Set	Meaning	When It Occurs
{PD}	High confidence for PD — singleton	$P(\text{PD})$ is high enough that only PD passes the threshold
{Healthy}	High confidence for Healthy — singleton	$P(\text{HC})$ is high enough that only HC passes
{Healthy, PD}	Ambiguous — both plausible	Neither $P(\text{PD})$ nor $P(\text{HC})$ is dominant enough
$\emptyset$ (empty)	Very rare — both scores too high	Edge case; treated as FLAG



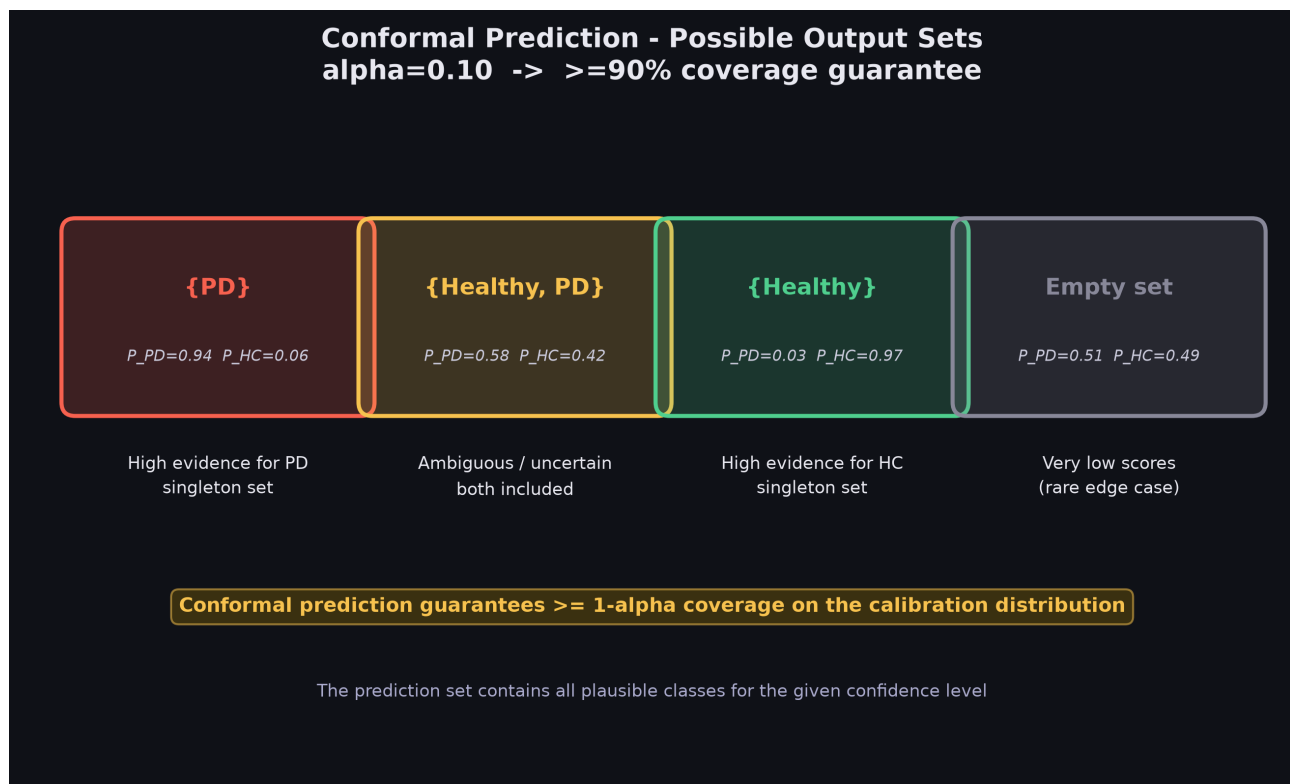
Conformal Coverage

Figure: Empirical coverage vs target coverage at $\alpha=0.10$ (dotted lines = 90% target).



Conformal Set Size

Figure: Average prediction set size per fold. Fold 3 has the most singletons (model is most decisive).



Conformal Explained

Figure: Visual explanation of the 4 possible conformal prediction output sets.

14. Reliability / Abstention Layer

The Problem

A model can make confident but wrong predictions — especially under cohort shift. The **Reliability Layer** acts as a gatekeeper: if the model's confidence, uncertainty, or conformal evidence suggests it is not reliable for this subject, it issues a **FLAG** to prompt human review instead of acting on the prediction.

How It Works — Three-Layer Check

The reliability layer runs three independent checks. **Any single flag triggers FLAG status.**

Step 1: Confidence Check

If $\max(P_{PD}, P_{HC}) < \tau_{\text{conf}} \rightarrow \text{FLAG (Low Confidence)}$

Step 2: Uncertainty Check

If $(1 - \max_{\text{prob}}) > \tau_{\text{unc}} \rightarrow \text{FLAG (High Uncertainty)}$

Step 3: Conformal Set Check

If $|\text{conformal_set}| > 1 \rightarrow \text{FLAG (Ambiguous Set)}$

If $|\text{conformal_set}| = 0 \rightarrow \text{FLAG (Empty Set)}$

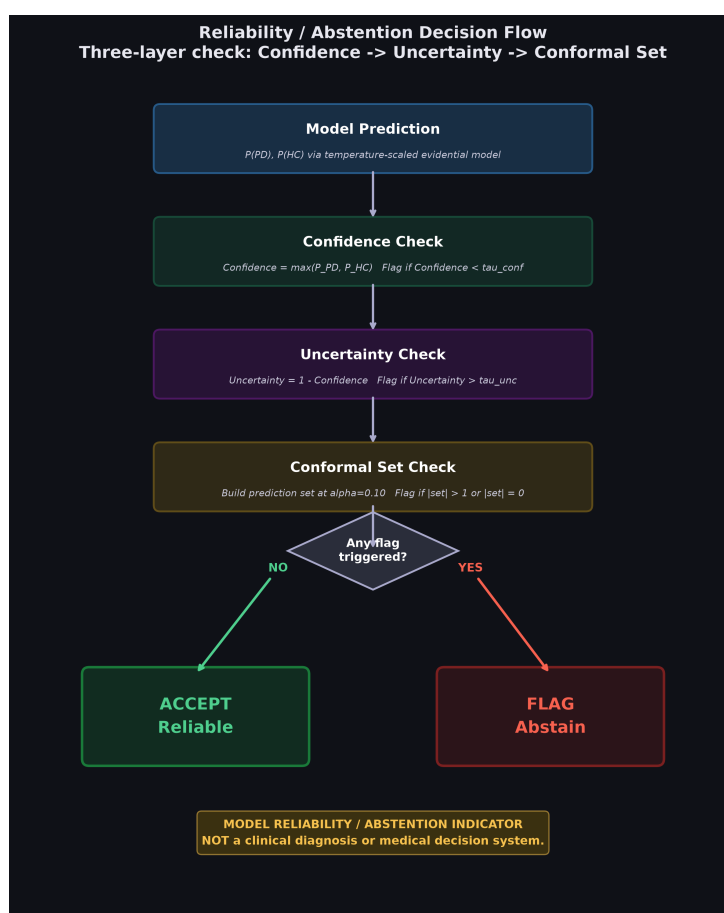
If $\text{conformal_set} \neq \{\text{point_prediction}\} \rightarrow \text{FLAG (Contradiction)}$

If all checks pass $\rightarrow \text{ACCEPT}$

Threshold Parameters Per Fold

Fold	Test	τ_{conf}	τ_{unc}	q_{hat}
1	Si	0.550	0.404	0.919
2	Ju	0.550	0.450	0.815
3	Ga	0.749	0.208	0.765

These thresholds are derived from the **validation set** using a percentile-based heuristic. They are frozen at inference time and stored in the conformal results CSV.



Reliability Decision Flow

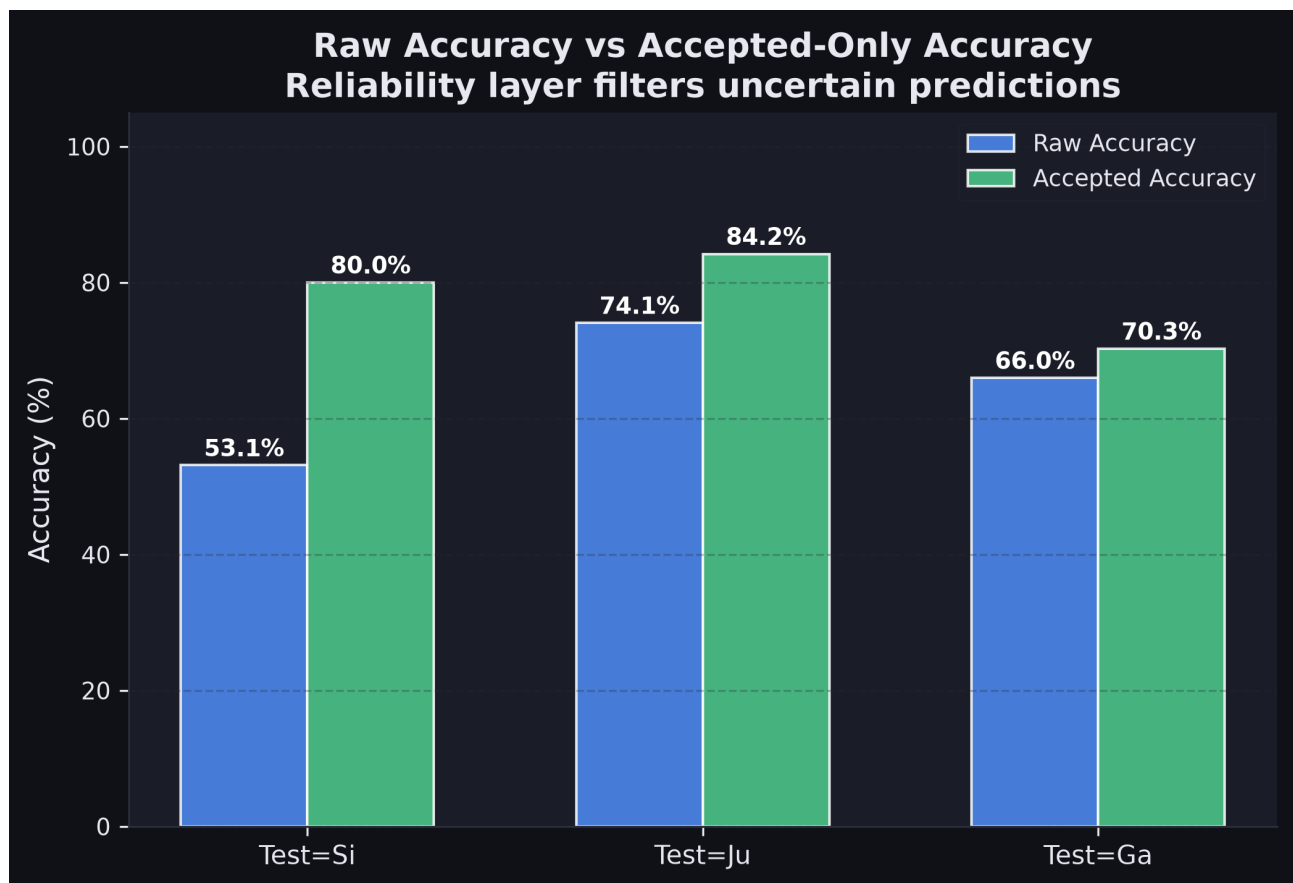
Figure: The three-layer ACCEPT/FLAG decision flow.

Reliability Results

Fold	Test	Raw Acc	Accepted Acc	Coverage	Error Capture
1	Si	53.1%	80.0%	15.6%	93.3%
2	Ju	74.1%	84.2%	35.2%	78.6%
3	Ga	66.0%	70.3%	78.7%	31.3%

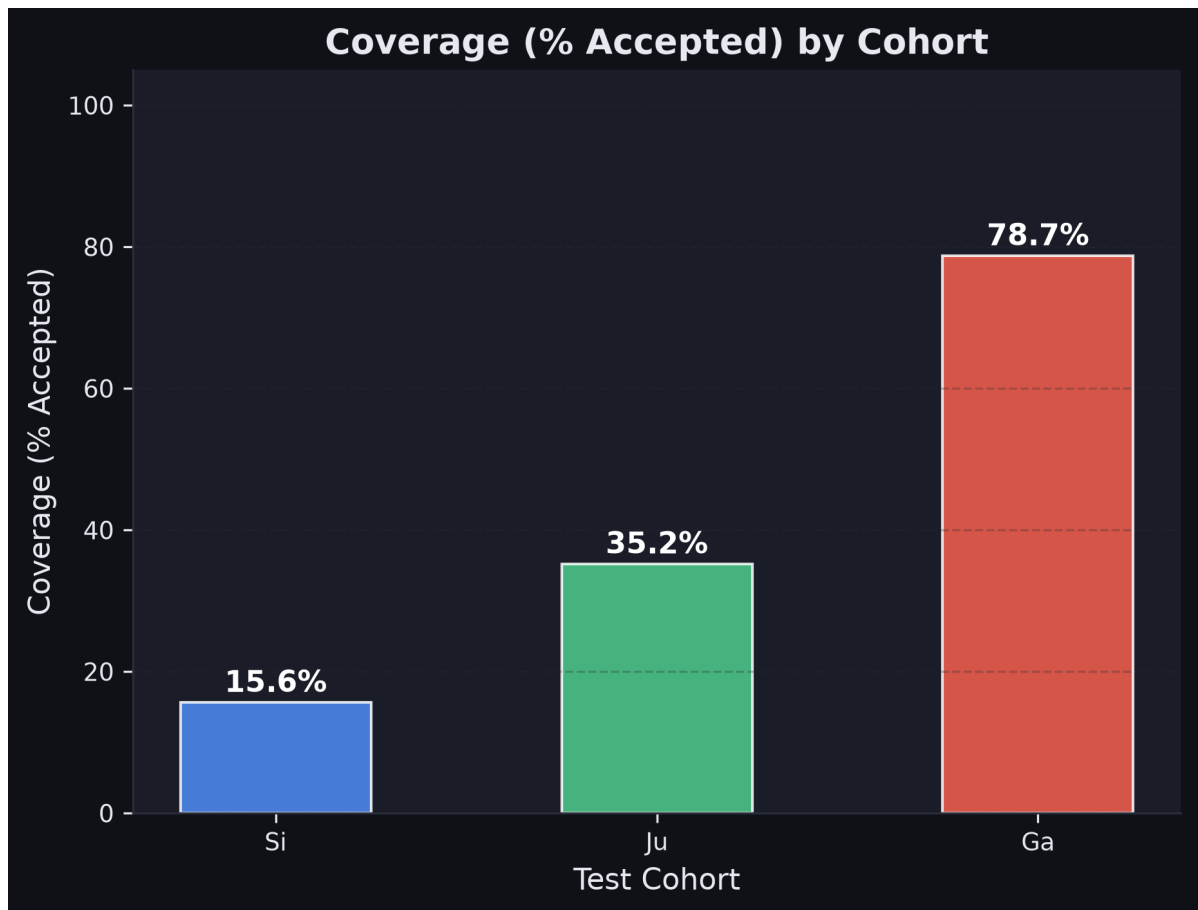
Definitions:

- **Coverage:** Fraction of test subjects accepted (not flagged)
- **Accepted Accuracy:** Accuracy on the accepted (non-flagged) subset
- **Error Capture Rate:** Fraction of the model's mistakes that were correctly flagged



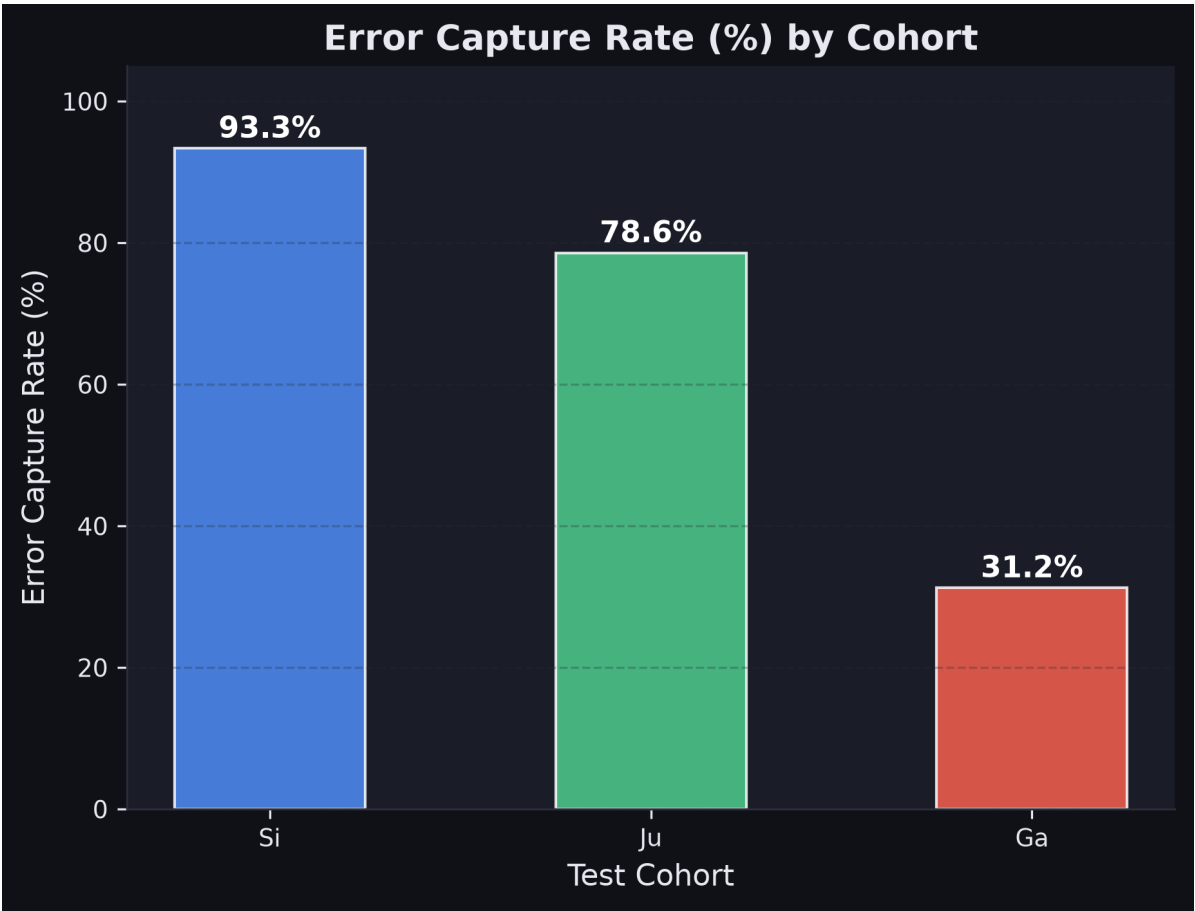
Raw vs Accepted Accuracy

Figure: Accepted-only accuracy is higher than raw accuracy in all 3 folds.



Coverage by Cohort

Figure: Fraction of subjects accepted by the reliability layer per cohort.



Error Capture

Figure: Fraction of model errors that were flagged and captured by the reliability layer.

Interpretation

Fold	Observation
→Si (Exp 1)	Very conservative — only 15.6% accepted, but among accepted, accuracy is 80% and 93.3% of errors are caught. The model knows it struggles with this cohort.
→Ju (Exp 2)	Moderate — 35.2% accepted. 84.2% accuracy on accepted; most errors captured.
→Ga (Exp 3)	Liberal — 78.7% accepted. The model is more confident on Ga subjects, but error capture drops (model doesn't know it's wrong).

Flag Reason Breakdown

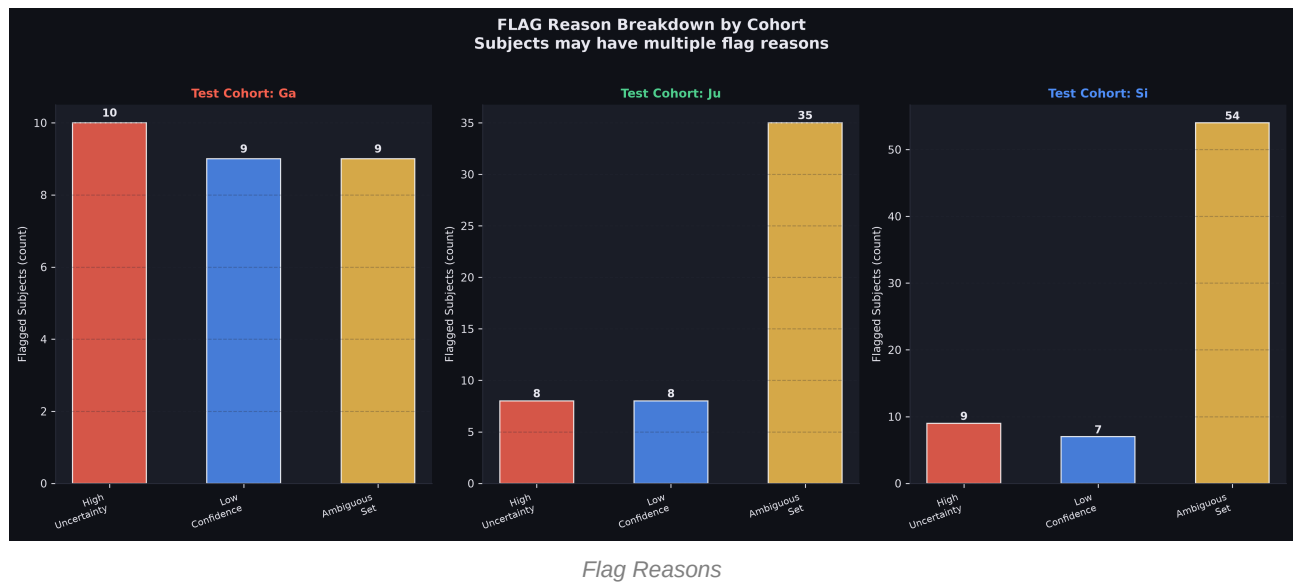


Figure: Which flag reasons trigger the most abstentions per cohort.

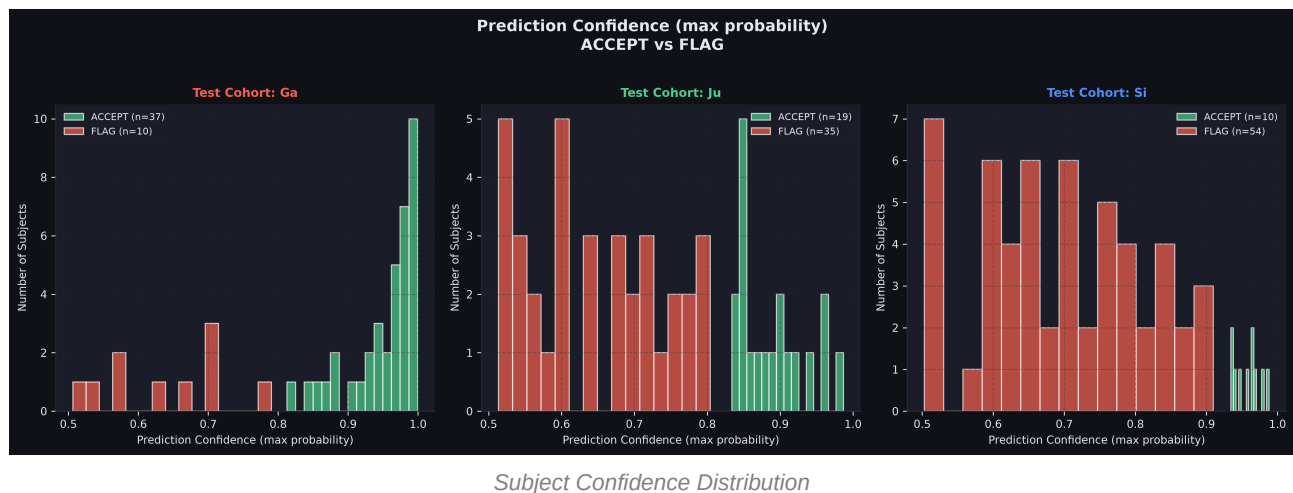


Figure: Confidence score distributions for ACCEPT vs FLAG subjects.

15. Corruption / Cohort-Shift Stress Testing

Motivation

In the real world, signals can be corrupted by sensor noise, calibration drift, or missing data. We simulate four types of corruption at 5 severity levels (0=clean, 4=severe) to test model robustness.

Corruption Types

Type	Description	Real-World Analogy
gaussian_noise	Add Gaussian noise to signal	Sensor measurement error
amplitude_scaling	Scale amplitude randomly per channel	Sensor drift / gain mismatch
temporal_masking	Zero out random time segments	Signal loss / packet drop
channel_dropout	Drop entire channels	Sensor failure / wire disconnection

What We Found

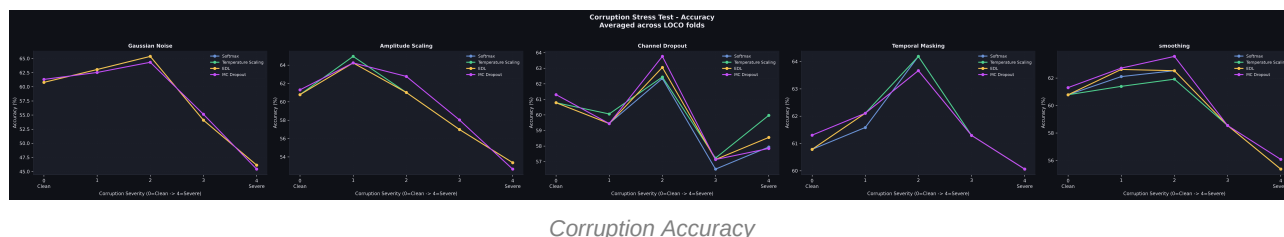


Figure: Accuracy degrades with severity, but the rate varies by corruption type and method.

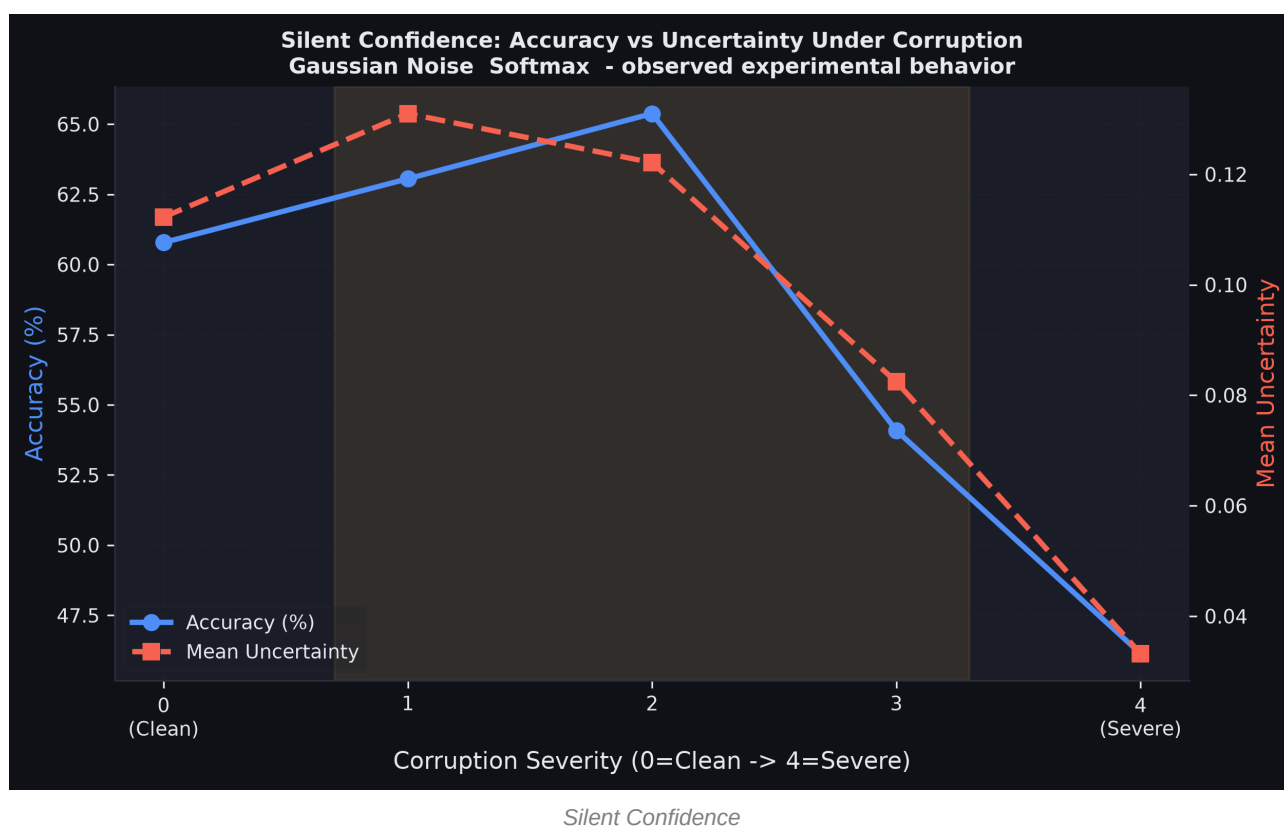


Figure: "Silent confidence" phenomenon — the Softmax model's accuracy drops but its uncertainty does not increase proportionally. This shows that raw Softmax is NOT a reliable uncertainty measure.

The "Silent Confidence" Problem

This is the core research finding for the Gait component:

As input corruption increases, a Softmax model becomes **less accurate** but does **not become proportionally more uncertain**. It remains falsely confident even when it's wrong.

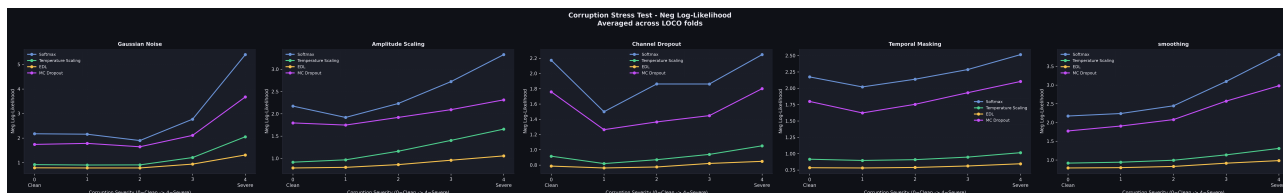
This motivates:

Temperature Scaling — improves base calibration

Deep Ensemble / MC Dropout — provides more honest uncertainty

Conformal Prediction — provides coverage guarantees

Reliability Layer — catches cases where confidence signals don't align



Corruption NLL

Figure: NLL under corruption — uncertainty-aware methods degrade more gracefully.

16. Individual Inference (Zero-Retraining Demo)

Demo Pipeline

```
python Gait/demo.py --recording path/to/recording.txt --fold 1
```

The demo runs without any retraining using frozen assets:

Asset	File
Normalization stats	Gait/checkpoints/normalization_stats.json
Fold 1 model (→Si)	Gait/checkpoints/deep_ensemble/ensemble_exp1_member0_seed42.pt
Fold 2 model (→Ju)	Gait/checkpoints/deep_ensemble/ensemble_exp2_member0_seed42.pt
Fold 3 model (→Ga)	Gait/checkpoints/deep_ensemble/ensemble_exp3_member0_seed42.pt
Calibration thresholds	Loaded from conformal_results.csv

Inference Flow



Individual Inference Flow

Figure: Zero-retraining inference pipeline for a single new gait recording.

Example Output

```

=== Gait PD Detection - Individual Inference ===
Recording   : GaSi_example.txt
Windows     : 47 windows (5-sec each, 50% overlap)
Fold        : 1 (trained on Ga+Ju, test cohort was Si)
  
```

Results:

```

Prediction      : Parkinson's Disease (PD)
P(PD)           : 0.782
P(HC)           : 0.218
Confidence      : 0.782
Uncertainty     : 0.218
Conformal Set   : {PD}
Reliability     : ACCEPT
  
```

Note: This is a research demonstration, not a clinical tool.

17. Repository Structure

```

Gait/
├── model.py
└── GaitCNNBiLSTM architecture
  
```

■■■ dataset.py	← GaitDataset, windowing, subject splitting
■■■ demo.py	← Individual inference demo
■	
■■■ experiments/	
■ ■■■ leave_one_cohort_out.py	← LOCO training & evaluation
■ ■■■ multiseed_loco.py	← Multi-seed (42-46) experiments
■ ■■■ calibration.py	← Temperature scaling
■ ■■■ edl_experiment.py	← Evidential Deep Learning
■ ■■■ mc_dropout.py	← Monte Carlo Dropout
■ ■■■ deep_ensemble.py	← Deep Ensemble (5 members)
■ ■■■ conformal_prediction.py	← Conformal prediction sets
■ ■■■ corruption_shift.py	← Corruption stress testing
■ ■■■ reliability_analysis.py	← Reliability / abstention layer
■	
■■■ checkpoints/	
■ ■■■ normalization_stats.json	← Frozen channel-wise mean/std
■ ■■■ deep_ensemble/	← 15 .pt files (3 folds × 5 seeds)
■	
■■■ outputs/	
■ ■■■ loco/	← loco_results.csv, subject_predictions.csv
■ ■■■ multiseed/	← multiseed_summary.csv
■ ■■■ calibration/	← calibration_results.csv
■ ■■■ deep_ensemble/	← deep_ensemble_results.csv, comparison.csv
■ ■■■ conformal/	← conformal_results.csv, corruption_coverage.csv
■ ■■■ corruption_shift/	← corruption_shift_results.csv
■ ■■■ reliability/	← reliability_results.csv, subject_decisions.csv
■	
■■■ visualizations/	
■ ■■■ generate_all_visualizations.py	← Generates all 45+ figures
■ ■■■ README.md	← Figure index
■ ■■■ architecture/	
■ ■■■ data_pipeline/	
■ ■■■ performance/	
■ ■■■ calibration/	
■ ■■■ uncertainty/	
■ ■■■ cohort_shift/	
■ ■■■ conformal/	
■ ■■■ reliability/	
■ ■■■ inference/	
■	
■■■ tests/	
■ ■■■ test_loco.py	
■ ■■■ test_reliability_layer.py	
■ ■■■ ...	
■	
■■■ gait-in-parkinsons-disease-1.0.0/	← PhysioNet dataset (309 .txt files)

18. How to Run

Prerequisites

```
pip install torch torchvision numpy pandas scikit-learn matplotlib tqdm
```

GPU with CUDA recommended. The training uses `torch.amp` (mixed precision) automatically when a GPU is available.

Step 1: Download Dataset

Place the PhysioNet Gait dataset in:

```
Gait/gait-in-parkinsons-disease-1.0.0/
```

Expected: 309 .txt files (*GaCo*, *GaPt*, *JuCo*, *JuPt*, *SiCo*, *SiPt*).

Step 2: Run LOCO Experiment (Single Seed)

```
python -m Gait.experiments.leave_one_cohort_out
```

Outputs saved to `Gait/outputs/loco/`.

Step 3: Multi-Seed LOCO (Seeds 42–46)

```
python -m Gait.experiments.multiseed_loco
```

Outputs saved to `Gait/outputs/multiseed/`.

Step 4: Calibration

```
python -m Gait.experiments.calibration
```

Outputs saved to `Gait/outputs/calibration/`.

Step 5: Deep Ensemble (5 Models per Fold)

```
python -m Gait.experiments.deep_ensemble
```

Checkpoints saved to `Gait/checkpoints/deep_ensemble/`. Results to `Gait/outputs/deep_ensemble/`.

Step 6: Conformal Prediction

```
python -m Gait.experiments.conformal_prediction
```

Outputs saved to `Gait/outputs/conformal/`.

Step 7: Corruption Stress Test

```
python -m Gait.experiments.corruption_shift
```

Outputs saved to `Gait/outputs/corruption_shift/`.

Step 8: Reliability Analysis

```
python -m Gait.experiments.reliability_analysis
```

Outputs saved to Gait/outputs/reliability/.

Step 9: Generate All Visualizations

```
python Gait/visualizations/generate_all_visualizations.py
```

Saves 45 PNG/SVG figures to Gait/visualizations/.

Step 10: Run Regression Tests

```
python -m pytest Gait/tests -q
```

Individual Inference Demo

```
python Gait/demo.py --recording path/to/recording.txt --fold 1
```

19. Complete Results Reference

LOCO Baseline (Seed 42)

Exp	Train	Test	Acc	BalAcc	Sens	Spec	F1	AUC	NLL	Brier
1	Ga+Ju	Si	0.500	0.472	0.771	0.172	0.628	0.602	0.687	0.248
2	Ga+Si	Ju	0.685	0.674	0.828	0.520	0.738	0.783	0.595	0.206
3	Ju+Si	Ga	0.638	0.602	0.759	0.444	0.721	0.598	1.075	0.293

Temperature Scaling Calibration

Fold	Test	T	Softmax NLL	Cal NLL	Softmax ECE	Cal ECE
1	Si	1.815	0.996	0.774	0.307	0.231
2	Ju	5.757	1.155	0.573	0.259	0.087
3	Ga	10.510	4.370	1.399	0.320	0.282

Deep Ensemble

Fold	Test	Members	Acc	BalAcc	AUC	F1	NLL	Brier	ECE
1	Si	5	0.781	0.773	0.803	0.811	0.602	0.181	0.109
2	Ju	5	0.722	0.717	0.807	0.754	0.544	0.182	0.067
3	Ga	5	0.681	0.647	0.612	0.754	2.396	0.278	0.243

Conformal Prediction (alpha = 0.10)

Fold	Test	q_hat	Empirical Coverage	Avg Set Size	Singleton Rate	Ambiguity Rate
1	Si	0.919	96.9%	1.844	15.6%	84.4%
2	Ju	0.815	94.4%	1.648	35.2%	64.8%
3	Ga	0.765	76.6%	1.191	80.9%	19.1%

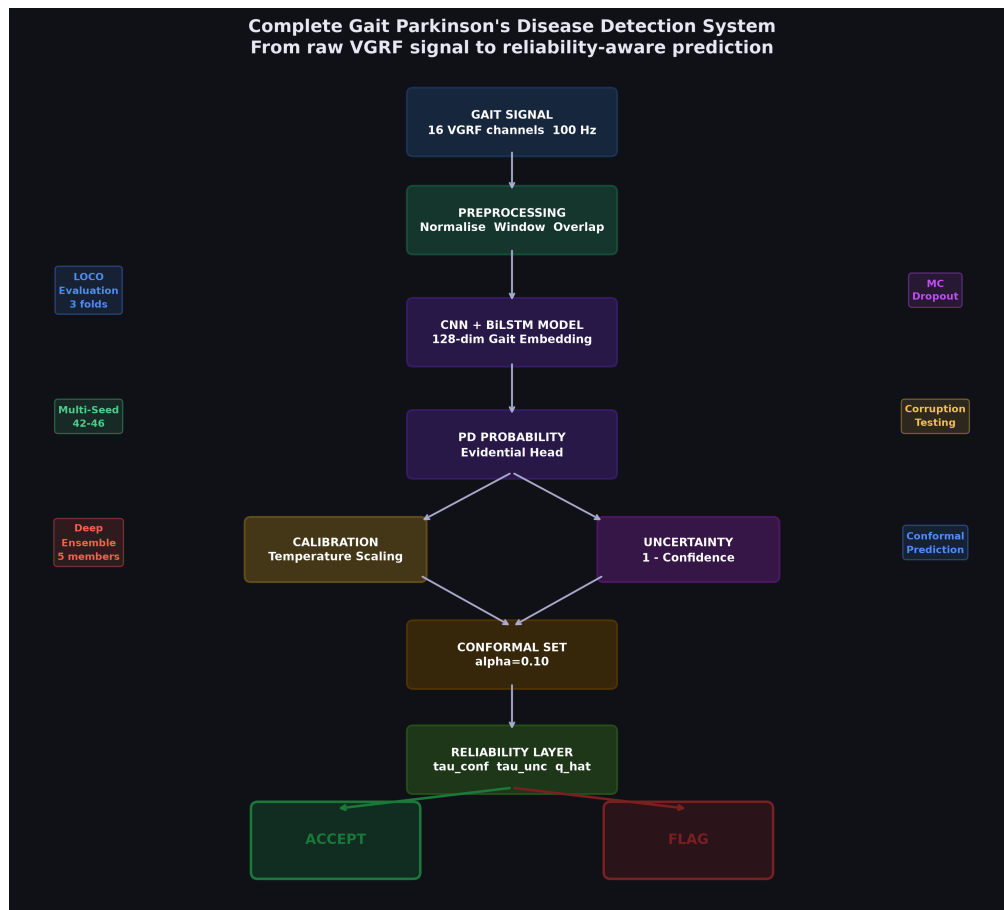
Reliability Layer

Fold	Test	Raw Acc	Accepted Acc	Coverage	Num Accepted	Num Flagged	Error Capture
1	Si	53.1%	80.0%	15.6%	10/64	54/64	93.3%
2	Ju	74.1%	84.2%	35.2%	19/54	35/54	78.6%
3	Ga	66.0%	70.3%	78.7%	37/47	10/47	31.3%

Multi-Seed Summary (5 Seeds, 95% CI)

Test Cohort	Metric	Mean	Std	CI Lower	CI Upper
Si	Accuracy	67.5%	8.80%	56.6%	78.4%
Si	AUC	0.735	0.068	0.650	0.819
Si	F1	0.736	0.047	0.678	0.794
Ju	Accuracy	71.9%	4.79%	65.9%	77.8%
Ju	AUC	0.784	0.046	0.727	0.840
Ju	F1	0.749	0.052	0.685	0.813
Ga	Accuracy	64.3%	2.33%	61.4%	67.1%
Ga	AUC	0.620	0.047	0.562	0.678
Ga	F1	0.739	0.004	0.735	0.744
All	Accuracy	67.9%	6.37%	64.3%	71.4%
All	AUC	0.713	0.087	0.665	0.761
All	F1	0.741	0.038	0.721	0.762

System Overview Figures



Final System Overview

Figure: Complete system — from raw VGRF signal through the full reliability-aware pipeline.



End-to-End Pipeline

Figure: End-to-end pipeline from raw recording to ACCEPT/FLAG decision.



Project Story

Figure: Five-stage project narrative — Detect → Generalise → Calibrate → Uncertainty → Abstain.

Important Notes & Limitations

This is a final-year project research system, NOT a clinical tool.

Not medically validated — results cannot be used for clinical diagnosis

Cohort shift is real — performance varies significantly across cohorts (50% to 78% accuracy)

Coverage trade-off — the reliability layer achieves higher accuracy only by refusing to predict on many subjects (especially in the $\rightarrow$ Si fold)

Uncertainty is not perfectly calibrated — particularly under cohort shift, models can remain overconfident

Dataset size — 165 subjects across 3 cohorts is relatively small; clinical deployment would need much more data

Walking conditions — all recordings were on flat surfaces under controlled conditions; real-world gait is more variable

Generated: 2026-10-03

All numbers from experimental outputs in Gait/outputs/. No data fabricated.